AcmeCorp Collections Agent & Bank Reconciliation

Explanation of algorithms, architecture and tech stack - Firoz Ahmad, Sonic AI Software Engineering Intern assignment

1. What was built

A web application for AcmeCorp's Accounts Team with two workflows. **Collections**: the system emails customers about overdue invoices over SMTP, reads their replies over IMAP, answers routine questions with an LLM, and holds anything risky for a person. **Reconciliation**: an uploaded bank-statement CSV is matched to invoices locally (no LLM); the system suggests an invoice with a 0-100 score and a one-line reason, and a person approves, rejects or overrides each row. Invoice balances change only on approval.

2. Architecture and tech stack

React + Vite front end (pages for organization, customers, invoices, collections, reconciliation) that talks only to a **FastAPI** back end over JSON; it holds no secrets. The back end is layered: *routes* (HTTP, validation) → *services* (use cases: agent, reconciliation, email, LLM, seeding) → *logic* (pure rules: money, overdue/TDS, matching, validation) and *repositories* (the only MongoDB code). **MongoDB** (pymongo) stores organization, customers, invoices, conversations, messages, claims, reminders, statements, transactions, payments and a Message-ID dedupe set; money uses Python `Decimal`. **rapidfuzz** supplies Jaro-Winkler and Levenshtein. The LLM client works with any OpenAI-compatible API or Anthropic through plain HTTP, so no vendor SDK is required. Tests use pytest, mongomock, a mocked LLM endpoint and a real local SMTP server (aiosmtpd).

3. Collections agent

Reminders. Overdue = status *sent*, balance > 0 and due date before today (Asia/Kolkata). One email per customer lists each overdue invoice with number, balance, due date and days overdue, plus the payment details and TDS policy. Text is produced by a deterministic template from the database, never by the LLM. The same invoice is not re-reminded within N days, and customers still on placeholder example.com addresses are skipped.

Inbound processing (each email, in order):

- **Dedupe and loop guard**: unique Message-ID record; ignore our own address; detect bounces and out-of-office mail (Auto-Submitted/Precedence/X-Autoreply headers, MAILER-DAEMON, subject and body patterns) and log them without replying; cap automatic replies per conversation per day.
- **Identify the customer** by matching the From address to the stored customer email (unique per customer). Unknown senders: no customer data is ever loaded; a holding draft goes to Needs attention. SPF/DKIM/DMARC failures are escalated as possible spoofing. Replies attach to the conversation through In-Reply-To/References, but never to another customer's conversation.
- **Fact sheet**: only that customer's *sent, unpaid* invoices (drafts and paid invoices are excluded) with amounts and dates pre-formatted (INR lakh grouping, e.g. 1,18,000.00), TDS amounts, organization bank details and policies. This is the only data the model sees.
- **Classification**: the LLM returns JSON (intents, confidence, claimed payment, draft reply). Intents: balance query, payment instructions, payment claim, TDS query (routine) versus extension, dispute, legal, other-company, unclear (need a person). The email is delimited as untrusted data in the prompt. A deterministic keyword rule set also runs and the final intent set is the *union*, so a model that calls a legal threat routine is overruled. Mixed emails (routine + extension) escalate entirely. Confidence below 0.7 or an LLM outage also escalates.
- **Validation before sending**: every currency amount must be in the customer's allowed amounts (balance, total, TDS, net-of-TDS, per-currency totals); every date must be an invoice/due date, today or a date the customer wrote; every invoice number must be an outstanding invoice of this customer; an invoice number must be followed by its own amount and due date; no other customer's name may appear. A failed check never sends: the row goes to Needs attention with a verified template draft and the rejected text is kept for audit.
- **Payment claims** ("I paid INV-1004, ref N123") are recorded with invoice numbers, claimed date and reference, and acknowledged, but never change an invoice: only a bank row approved by a person does.
- **Human review**: the person edits the draft and sends it; edits are re-validated (override requires an explicit "send anyway"); sends are atomic so a draft cannot go twice; replies always go to the address on record, never Reply-To.

4. Bank reconciliation algorithm

The CSV is parsed defensively (BOM, bad dates/amounts and unsupported currencies become *invalid* rows, size and row limits). Debits are ignored; bank interest/charge credits are labelled non-customer. For each credit row the engine runs these steps:

- **Narration parsing**: method (NEFT/RTGS/IMPS/UPI/SWIFT/SEPA), invoice references (INV 1004, INV1021, INV-1002, RE 1024, lists), PO numbers, part-payment keywords, and the remaining *payer text* after removing memo words.
- **Payer similarity** against each customer billed in the receipt's currency, combining soft token matching (exact, Jaro-Winkler for typos, Levenshtein, prefix such as TECH→Technologies, abbreviation-subsequence such as PKG→Packaging), weighted coverage of the customer's name (brand word counts double), precision of the narration, character tri-gram TF-IDF cosine and token-set ratio. If the brand word is not found the score is multiplied by 0.6, so generic words ("Clinic", "Retail") cannot carry a

match: "R K SHARMA CLINIC" scores low against "Redwood Clinics".

- **Candidate generation** from the open (sent, positive-balance) invoices of plausible customers: single invoices; every combination of 2-4 invoices (one payment for several invoices, e.g. INR 1,94,700 = INV-1010+1011+1012); and the explicitly referenced set. Each candidate gets an *amount case*: exact balance; balance less 10% TDS (10% of subtotal, also of the discounted subtotal); part payment; over-payment/mismatch (only kept when an invoice is named).
- **Scoring** by adding log-odds evidence to a prior: invoice number in narration (+5), payer similarity (-2.5 to +4), PO number (+4), customer email quoting the same bank reference (+5) or the invoice (+1.5), exact/TDS/multi amount (+3.5 to +4), part payment (+0.2, more with the keyword or a small foreign shortfall that looks like bank charges), plausible date (+0.5) versus payment before the invoice date (-14), identical-amount rivals (-0.8 each, then earliest due date wins). The sum is mapped through a logistic function to 0-99. Caps prevent over-confidence: unknown payer without any reference ≤ 45; part payment with a weak payer ≤ 49; over-payment ≤ 49. A score of 50 or more is a suggestion; lower shows "no confident match" with the closest candidate in the reason.
- **Global assignment**: rows are processed from highest to lowest confidence against virtual balances, so two receipts of the same amount are spread over different invoices instead of both claiming one.
- **Why not an LLM or embeddings?** Matching has to be deterministic, auditable and cheap, and amounts and invoice numbers are exact facts. Fuzzy string metrics on short bank narrations handle typos and truncation well, and every score component can be explained in the reason. A sentence-embedding model would add weight and opacity for little gain on company-name abbreviations; it could be added as one more evidence term.

Results on the sample statement: Kestrel→INV-1004 (not INV-1005, same amount); Orbis part payment→INV-1013 (66, medium); Quillon→INV-1007 with TDS; Nordlys→INV-1024 (INV-1025 is dated after the payment); Harbour→INV-1021; Brightline→INV-1002; Redwood→INV-1016; Thornfield→INV-1026; Saffron→INV-1010+1011+1012; R K Sharma, Maple Retail and the interest credit show no match; debits are ignored.

5. Approval and data integrity

Approval checks currency equality, that each invoice is outstanding, that allocations equal the receipt amount and never exceed a balance. A shortfall needs an explicit choice: *Close invoice (TDS deducted)* sets the balance to zero, status paid and records `tds_deducted` (a warning appears if it differs from the expected 10%), or *Leave balance open* reduces the balance. Status changes use atomic compare-and-set updates (pending→approving→approved), invoice updates are guarded on the previous balance with rollback if any allocation fails, and each approval writes an audit record. Re-uploading a statement flags rows already approved as duplicates. After approval the invoice drops out of reminders and out of the agent's context automatically because both read live invoice data.

6. Security, testing and limits

Signed expiring tokens, login rate limiting, authenticated routes, restricted CORS, typed validation (no Mongo operator injection), upload limits, plain-text mail only, CR/LF stripping, HTML stripping of inbound mail, secrets only in environment variables, prompt-injection containment (only own-customer facts in the prompt plus output validation), loop and duplicate protection, and non-leaky errors. 65 automated tests cover matching variants, approvals, reminders, replies, validation, escalation, injection and isolation, authentication and a real SMTP round trip. **Not verified** in my build environment: a live MongoDB server, a live IMAP mailbox and a live LLM (covered by mocks); these follow the README run steps. **Limitations**: no embeddings, no undo for approved payments, single admin login, IMAP polling instead of push.